SET 22

PART A — Short questions and answers

Q1. Summarize the steps to convert the DONOR entity with its attributes into a relation.

Answer: This concept helps organize, retrieve, or maintain database information. Use the example named in the question to explain it in one or two sentences.

Q2. Compare the UPDATE command and the ALTER command.

Answer: This concept helps organize, retrieve, or maintain database information. Use the example named in the question to explain it in one or two sentences.

Q3. Interpret the functional dependency Donor_ID → Blood_Group, City.

Answer: Book_ID → Title means each Book_ID determines exactly one book title.

Q4. Explain the purpose of the INTERSECT operator.

Answer: This concept helps organize, retrieve, or maintain database information. Use the example named in the question to explain it in one or two sentences.

Q5. Explain how GROUP BY divides the donation records into groups before SUM is applied.

Answer: GROUP BY collects rows with the same value so aggregate functions can calculate a result for each group.

Q6. Compare BEFORE and AFTER triggers.

Answer: This concept helps organize, retrieve, or maintain database information. Use the example named in the question to explain it in one or two sentences.

Q7. Explain why a graph database is preferred for a donor network.

Answer: A graph database stores entities as nodes and their connections as relationships.

Q8. Explain the function of the distinct() method.

Answer: This concept helps organize, retrieve, or maintain database information. Use the example named in the question to explain it in one or two sentences.

Q9. Interpret the MongoDB query: db.donors.find({BloodGroup: "O+", City: "Nellore"})

Answer: The query returns all documents that satisfy the specified filter condition.

Q10. Interpret the Cypher query: MATCH (d:Donor {bloodGroup: 'O+'}) RETURN d.name;

Answer: This concept helps organize, retrieve, or maintain database information. Use the example named in the question to explain it in one or two sentences.

PART B — System solution

Scenario: A regional blood bank maintains a Blood Bank Management System to store donor details, hospital

1(a) ER design:

• **DONOR**(DonorID, DonorName, BloodGroup, Age, City); primary key: **DonorID**.

• **HOSPITAL**(HospitalID, HospitalName, City); primary key: **HospitalID**.

• **DONATION**(DonationID, DonorID, HospitalID, DonationDate, Units); primary key: **DonationID**.

Connect the two main entity tables through the transaction/relationship table. The transaction table has its own primary key and foreign keys to both entity tables. Cardinality: each entity can participate in many transactions; each transaction belongs to one row from each entity table (1:M on both sides, resolving M:N). Include the transaction's date/status/amount attributes shown in the question.

1(b) SQL DDL/DML: Create each table with its listed columns, PRIMARY KEY on its ID, and FOREIGN KEYs in the transaction table. Add NOT NULL to required fields; use CHECK/UNIQUE/DEFAULT where the question asks. Insert five sample rows per table, making sure foreign-key values exist.

```
CREATE TABLE DONOR (
  DonorID NUMBER PRIMARY KEY,
  DonorName VARCHAR2(100),
  BloodGroup VARCHAR2(100),
  Age NUMBER,
  City VARCHAR2(100)
);
CREATE TABLE HOSPITAL (
  HospitalID NUMBER PRIMARY KEY,
```

```
  HospitalName VARCHAR2(100),
  City VARCHAR2(100)
);
CREATE TABLE DONATION (
  DonationID NUMBER PRIMARY KEY,
  DonorID NUMBER,
  HospitalID NUMBER,
  DonationDate DATE,
  Units NUMBER
);
```

2(a) Join query: Select the two entity names/details from the transaction table by joining the transaction table to both entity tables using their IDs.

```
SELECT a.*, b.*
FROM DONATION x
JOIN DONOR a ON x.DonorID = a.DonorID
JOIN HOSPITAL b ON x.HospitalID = b.HospitalID;
```

2(b) Aggregate query: Group the transaction rows by the requested category/type and apply HAVING to the aggregate condition stated in the question.

```
SELECT grouping_column, COUNT(*) AS total
FROM transaction_table
GROUP BY grouping_column
HAVING COUNT(*) > threshold;
```

2(c) PL/SQL procedure: Create a procedure with an IN parameter for the requested ID. Join the transaction table to the related entity tables, filter by that parameter, and print selected values using DBMS_OUTPUT.PUT_LINE.

```
CREATE OR REPLACE PROCEDURE show_details(p_id IN NUMBER) AS
BEGIN
  FOR r IN (
    SELECT a.*, b.*, x.*
    FROM transaction_table x
    JOIN entity_table_1 a ON x.entity1_id = a.entity1_id
    JOIN entity_table_2 b ON x.entity2_id = b.entity2_id
    WHERE x.transaction_id = p_id
  ) LOOP
    DBMS_OUTPUT.PUT_LINE('Details retrieved');
  END LOOP;
END;
/
```

3. Graph database (Neo4j/Cypher): Create the required node labels and relationship types exactly as specified in the set. Example pattern:

```
CREATE (a:EntityA {id: 1, name: 'Name A'});
CREATE (b:EntityB {id: 2, name: 'Name B'});
MATCH (a:EntityA {id: 1}), (b:EntityB {id: 2})
CREATE (a)-[:RELATED_TO]->(b);
MATCH (a:EntityA)-[:RELATED_TO]->(b:EntityB)
RETURN a.name, b.name;
```